DSA

1. Longest Subarray with Sum K Find the maximum length contiguous subarray with sum K; values may be negative

Code:

```
class Solution:
  def longestSubarray(self, arr, k):
    # code here
    p=0
    first={}
    ans=0
    for i in range(len(arr)):
      p+=arr[i]
      if p==k:
        ans=i+1
      if p-k in first:
        ans=max(ans,i-first[p-k])
      if p not in first:
        first[p]=i
    return ans
```

2. Minimum Window Containing All Required Skills Return the smallest substring of S containing all characters of T with required frequencies.

Code:

3. Merge Overlapping Intervals Merge overlapping meeting intervals and return the resulting non-overlapping schedule.

Code:

```
class Solution:
  def merge(self, intervals: List[List[int]]) -> List[List[int]]:
    intervals.sort(key=lambda x:x[0])
    merged=[]
    for interval in intervals:
      if not merged or merged[-1][1]<interval[0]:
        merged.append(interval)
      else:
        merged[-1][1]=max(merged[-1][1],interval[1])
    return merged
```

4. Kth Largest Element in a Stream After each insertion, output the kth largest value once at least k values exist

Code:

```
import heapq


class KthLargest:
  def __init__(self, k: int, nums: List[int]):
    self.k = k
    self.heap = nums
    heapq.heapify(self.heap)
    while len(self.heap) > k:
      heapq.heappop(self.heap)
  def add(self, val: int) -> int:
    heapq.heappush(self.heap, val)
    if len(self.heap) > self.k:
      heapq.heappop(self.heap)
    return self.heap[0]
```

5. Detect and Remove Cycle in Linked List Detect a cycle and remove it without losing any nodes

Code:

```
class Solution:
  def detectCycle(self, head: Optional[ListNode]) -> Optional[ListNode]:
    slow=fast=head
    while fast and fast.next:
      fast=fast.next.next
      slow=slow.next
      if slow==fast:
        slow=head
        while slow!=fast:
          slow=slow.next
          fast=fast.next
        return slow
    return None
```

6. Lowest Common Ancestor in Binary Tree Return the lowest common ancestor of two existing nodes in a binary tree

Code:

```
class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        if root==None or root==p or root==q:
            return root
        left=self.lowestCommonAncestor(root.left,p,q)
        right=self.lowestCommonAncestor(root.right,p,q)
        if left and right:
            return root
        if left:
            return left
        return right
```

7. Shortest Path in a Binary Matrix Find shortest top-left to bottom-right path through open cells using 8 directions

Code:

```
class Solution:
  def shortestPathBinaryMatrix(self, grid: list[list[int]]) -> int:
    n=len(grid)
    if grid[0][0]==1 or grid[n-1][n-1]==1:
      return -1
    q=deque()
    q.append((0,0,1))
    visited={(0,0)}
    di=[(-1,-1),(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0),(1,1)]
    while q:
      r,c,d=q.popleft()
      if r==n-1 and c==n-1:
        return d
      for dr,dc in di:
        nr=r+dr
        nc=c+dc
        if 0<=nr<n and 0<=nc<n and grid[nr][nc]==0 and (nr,nc) not in visited:
          visited.add((nr,nc))
          q.append((nr,nc,d+1))
    return -1
```

8. Course Completion Order Return one valid course order from prerequisite pairs, or empty if impossible

Code:

```
class Solution:
  def findOrder(self, numCourses: int, prerequisites: list[list[int]]) -> list[int]:
    graph=[[] for i in range(numCourses)]
    indegree=[0]*numCourses
    for course,pre in prerequisites:
      graph[pre].append(course)
      indegree[course]+=1
    q=deque()
    for i in range(numCourses):
      if indegree[i]==0:
        q.append(i)
    result=[]
    while q:
      course=q.popleft()
      result.append(course)
      for nxt in graph[course]:
        indegree[nxt]-=1
        if indegree[nxt]==0:
          q.append(nxt)
    if len(result)==numCourses:
      return result
    return []
```

9. Minimum Coins for Amount Find the minimum coins required for a target amount, or -1 if impossible

Code:

```
class Solution:
  def coinChange(self, coins: List[int], amount: int) -> int:
    dp=[float('inf')]*(amount+1)
    dp[0]=0
    for i in range(1,amount+1):
      for coin in coins:
        if i-coin>=0:
          dp[i]=min(dp[i],1+dp[i-coin])
    if dp[amount]==float('inf'):
      return -1
    return dp[amount]
```

10. Range Maximum Query with Updates Support point updates and maximum queries over an inclusive range [L,R].

Code: